

Rajalakshmi Engineering College

Name: Naren S

Email: 241901066@rajalakshmi.edu.in

Roll no: 241901066

Phone: 6382463115

Branch: REC

Department: CSE (CS) - Section 1

Batch: 2028

Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, InvalidAmountException and InsufficientFundsException, both extending the Exception class. Throw an InvalidAmountException with a message if the deposit amount is less than or equal to zero. Throw an InsufficientFundsException if the withdrawal amount is greater than the available balance. Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value B, representing the initial balance.

The second line consists of a double value D, representing the deposit amount.

The third line consists of a double value W, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an `InvalidAmountException` occurs, print "Error: [D] is not valid".

If an `InsufficientFundsException` occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1

270.2

150.3

Output: Amount Withdrawn: 150.3

Current Balance: 1170.0

Answer

```
import java.util.*;
```

```
class InvalidAmountException extends Exception {  
    public InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

```
class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double balance = sc.nextDouble();
        double deposit = sc.nextDouble();
        double withdraw = sc.nextDouble();

        try {
            if (deposit <= 0) {
                throw new InvalidAmountException(deposit + " is not valid");
            }
            balance += deposit;

            if (withdraw > balance) {
                throw new InsufficientFundsException("Insufficient funds");
            }

            balance -= withdraw;
            System.out.printf("Amount Withdrawn: %.1f Current Balance: %.1f",
                withdraw, balance);
        }
        catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

In an online shopping cart system, users can apply coupon codes during checkout to avail of discounts. However, to ensure the validity and security of coupon codes, the system enforces specific rules for their format. Your

task is to implement a Java program named `CouponCodeValidator` that takes user input for a coupon code and validates it according to the specified rules.

Rules for Valid Coupon Code:

The coupon code must consist of exactly 10 characters. The coupon code must contain at least one alphabet (uppercase or lowercase) and at least one digit (0-9). Special characters are not allowed in the coupon code.

Implement a custom exception, `InvalidCouponException`, to handle cases where the entered coupon code does not meet the specified criteria.

Input Format

The input consists of a string `s`, representing the coupon code.

Output Format

The output is displayed in the following format:

If the entered coupon code meets the specified criteria, the program outputs

"Coupon code applied successfully!"

If the entered coupon code has less than or more than 10 characters it outputs

"Error: Invalid coupon code length. It must be exactly 10 characters."

If the entered coupon code contains only numeric or only alphabets it outputs

"Error: Invalid coupon code format. It must contain at least one alphabet and one digit."

If the entered coupon code contains special characters it outputs

"Error: Coupon code should not contain special characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ABCD123456

Output: Coupon code applied successfully!

Answer

```
import java.util.*;
```

```
class InvalidCouponException extends Exception {  
    public InvalidCouponException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String s = sc.nextLine();  
        try {  
            validateCoupon(s);  
            System.out.println("Coupon code applied successfully!");  
        } catch (InvalidCouponException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

```
    public static void validateCoupon(String s) throws InvalidCouponException {  
        if (s.length() != 10)  
            throw new InvalidCouponException("Error: Invalid coupon code length. It  
must be exactly 10 characters.");  
        if (!s.matches("[A-Za-z0-9]+"))  
            throw new InvalidCouponException("Error: Coupon code should not  
contain special characters.");  
        if (s.matches("[A-Za-z]+" ) || s.matches("[0-9]+"))  
            throw new InvalidCouponException("Error: Invalid coupon code format. It  
must contain at least one alphabet and one digit.");  
    }  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Theo is trying to update his payment information on a subscription-based streaming service. To proceed, the system requires Theo to provide a valid credit card number consisting of 16 digits. However, Theo wants to make sure that the credit card number he enters meets the specified criteria with proper exception handling.

The credit card number must consist of exactly 16 digits. If the entered credit card number does not meet the specified criteria, the program should throw a custom exception, `InvalidCreditCardException`, and provide Theo with specific error messages: If the length of the credit card number is not 16 digits, the exception message should be: "Invalid credit card number length." If the credit card number contains non-numeric characters, the exception message should be: "Invalid credit card number format."

Implement a custom exception, `InvalidCreditCardException`, to fulfill Theo's requirements and keep his payment information secure.

Input Format

The input consists of a string value 's', consisting of the 16-digit credit card number.

Output Format

The output is displayed in the following format:

If the entered credit card number is valid, the program should output a success message:

"Payment information updated successfully!"

If the entered credit card has more than 16 digits or less than 16 digits it displays

"Error: Invalid credit card number length."

If the entered 16-digit credit card has non-integers it displays

"Error: Invalid credit card number format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1234567890123456

Output: Payment information updated successfully!

Answer

```
import java.util.*;

class InvalidCreditCardException extends Exception {
    public InvalidCreditCardException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String cardNumber = sc.nextLine();

        try {
            if (cardNumber.length() != 16) {
                throw new InvalidCreditCardException("Invalid credit card number
length.");
            }

            for (char ch : cardNumber.toCharArray()) {
                if (!Character.isDigit(ch)) {
                    throw new InvalidCreditCardException("Invalid credit card number
format.");
                }
            }

            System.out.println("Payment information updated successfully!");
        }
        catch (InvalidCreditCardException e) {
            System.out.println("Error: " + e.getMessage());
        }

        sc.close();
    }
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Alice is designing a program that requires users to enter positive numbers. She wants to implement a solution that validates whether the entered number is positive. In case the input is not a positive number, she wants to throw a custom exception.

The number should be a positive integer. If this condition is violated, the program should throw a custom exception: `InvalidPositiveNumberException` with the message "Invalid input. Please enter a positive integer."

Implement a custom exception, `InvalidPositiveNumberException`, to handle cases where the entered number does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the entered number.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Number {number} is positive."

The {number} represents the entered positive integer.

If the entered number is negative then it displays

"Error: Invalid input. Please enter a positive integer."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

Output: Number 100 is positive.

Answer

```
import java.util.*;

class InvalidPositiveNumberException extends Exception {
    public InvalidPositiveNumberException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        try {
            validateNumber(n);
            System.out.println("Number " + n + " is positive.");
        } catch (InvalidPositiveNumberException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    public static void validateNumber(int n) throws
    InvalidPositiveNumberException {
        if (n <= 0)
            throw new InvalidPositiveNumberException("Invalid input. Please enter a
            positive integer.");
        }
    }
}
```

Status : Correct

Marks : 10/10